codingninjas

Class 2: Introduction to Continuous Integration

Session Overview

1. Principles of CI and its significance in frequent, automated code integration.
2. Overview of CI pipeline components: source control, build servers, and testing frameworks.
3. Practical setup of a Jenkins CI pipeline, including Jenkinsfile creation.
4. Best practices for effective CI implementation.

Prerequisites

- Basic understanding of software development and operations.
- Familiarity with Agile methodologies is a plus.

Introduction

Definition and Purpose of CI

Continuous Integration (CI) is a software development practice where developers frequently integrate their code into a shared repository. Typically, this integration occurs multiple times a day. Each integration is then verified by an automated build and testing process, allowing teams to detect and address issues early.

Purpose of CI in DevOps Lifecycle:

- **Early Error Detection:** CI enables the early detection of errors by integrating and testing code continuously. This reduces the risk of integration problems that could delay the software development lifecycle.
- **Faster Release Cycles:** With frequent integrations and automated testing, CI facilitates faster and more reliable release cycles. This is crucial in DevOps, where the goal is to achieve continuous delivery and deployment.
- **Reduced Integration Problems:** By integrating code frequently, CI minimizes the complexity of merging code changes, leading to fewer conflicts and easier resolution of issues.

Importance of Frequent Code Integration

Frequent integration of code is at the heart of CI. The primary goal is to reduce the integration problems that occur when code changes are merged infrequently, which can lead to complicated and time-consuming fixes.

Risks of Infrequent Integration:

- **Merge Conflicts:** The longer code remains unintegrated, the more likely it is to conflict with other developers' changes, making integration more difficult and error-prone.

- **Delayed Feedback:** Infrequent integration delays the detection of errors, leading to more complex and costly fixes later in the development cycle.

Benefits of Frequent Integration:

- **Constant Feedback:** Continuous integration provides immediate feedback on the health of the codebase, allowing developers to address issues as soon as they arise.
- **Early Bug Detection:** Frequent integration and testing catch bugs early, reducing the risk of defects making their way into production.
- **Improved Collaboration:** Developers work more collaboratively when integration is frequent, leading to a more cohesive and synchronized team effort.

2. CI Pipeline Components

A CI pipeline is a series of automated processes that ensure code changes are integrated, built, and tested continuously. The CI pipeline is a critical component of the CI/CD (Continuous Integration/Continuous Deployment) process.

Key Components of a CI Pipeline

1. **Source Control:**

Source control management (SCM) systems like Git are central to CI. They allow multiple developers to collaborate on a shared codebase, tracking changes, managing versions, and enabling easy rollbacks if necessary. Tools like GitHub and Bitbucket are commonly used to manage repositories in CI pipelines.

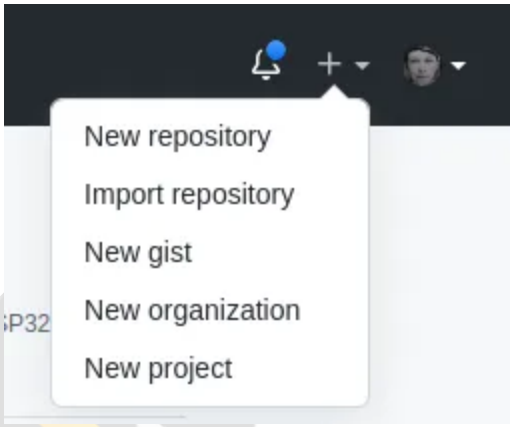
- All code changes are committed to a central repository (e.g., Git, SVN).
- Example: Using GitHub as the source control.
- Commands:

```
git init
git add .
git commit -m "Initial commit"
git push origin main
```

How to create Github repository and connecting local repository with Github

Login to your github account

Click on top right corner on + dropdown menu. Click on New repository.



Now give a new name to your repository and then click on create repository.

codingninjas

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere?
Import a repository.

Repository template
Start your repository with a template repository's contents.

No template

Owner * **Repository name** *

sleepypioneer / test_site

Great repository names are sl test_site is available. Need inspiration? How about **fuzzy-memory**?

Description (optional)

Public
Anyone on the internet can see this repository. You choose who can commit.

Private
You choose who can see and commit to this repository.

Initialize this repository with:
Skip this step if you're importing an existing repository.

Add a README file
This is where you can write a long description for your project. Learn more.

Add .gitignore
Choose which files not to track from a list of templates. Learn more.

Choose a license
A license tells others what they can and can't do with your code. Learn more.

Create repository

Now you will land on screen with following command

...or create a new repository on the command line

```
echo "# test_site" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:sleepypioneer/test_site.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin git@github.com:sleepypioneer/test_site.git
git branch -M main
git push -u origin main
```

Now set up a local git repository and commit all the changes.

Now take the last 3 commands from the github repository and run it in the project directory. If you are doing it for the first time then github will ask you for authentication and then push your code to Github.

After pushing done then your code from local will be visible on github repository.

2. **Build Server:**

The build server is the automation hub of the CI process. Jenkins is a widely used open-source automation server that facilitates the building, testing, and deployment of code. The build server continuously monitors the source control system for changes, triggers automated builds, and runs tests to validate the integration.

- A build server (like Jenkins) automates the process of compiling and packaging the code.
- The build server pulls the latest code from the repository, builds it, and runs tests.

3. **Testing Frameworks:**

Automated testing is integral to maintaining code quality in a CI pipeline. Testing frameworks like Jest or Mocha are used to run unit tests, integration tests, and sometimes even end-to-end tests. The results of these tests are crucial in determining whether the code is ready for deployment.

The Role of Automated Tests in CI

Automated tests play a vital role in CI by ensuring that each code change is validated before it is integrated into the main branch. This helps in maintaining code quality and stability.

- **Unit Tests:** These are the smallest and fastest tests, focusing on individual components or functions within the codebase.
- **Integration Tests:** These tests ensure that different modules or components work together as expected.
- **End-to-End Tests:** These tests simulate real user scenarios to validate the overall functionality of the application.

```
const request = require('supertest');
const app = require('../app'); // Assuming your Express app is in app.js

describe('GET /', () => {
  it('should return Hello, World!', async () => {
    const res = await request(app).get('/');
    expect(res.text).toBe('Hello, World!');
    expect(res.statusCode).toBe(200);
  });
});
```

The success of a CI pipeline heavily relies on the robustness and comprehensiveness of the automated tests it runs.

Setting Up a CI Pipeline Using Jenkins

Introduction to Jenkins:

- **Jenkins** is an open-source automation server that helps implement Continuous Integration (CI). It automates the build, test, and deployment processes, facilitating frequent integration of changes in the project.

Step-by-Step Guide to Setting Up a Jenkins CI Pipeline

1. Install Jenkins:

- You can install Jenkins on a local machine or a cloud instance such as AWS EC2.
- **Commands for Ubuntu**:

```
sudo apt update
sudo apt install openjdk-11-jdk -y
wget -q -O - https://pkg.jenkins.io/debian/jenkins.io.key | sudo apt-key add -
sudo sh -c 'echo deb http://pkg.jenkins.io/debian-stable binary/ >
/etc/apt/sources.list.d/jenkins.list'
sudo apt update
sudo apt install jenkins -y
sudo systemctl start jenkins
```

2. Setting Up Jenkins:

- Access Jenkins at `http://localhost:8080`.
- Unlock Jenkins using the initial admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

- Follow the setup process and install the suggested plugins.

3. Creating a Jenkins Freestyle Project:

- **Go to**: `New Item` > `Freestyle Project`.
- **Configure the project** to pull code from a Git repository:

```
git clone https://github.com/your-repo.git
```

Add build steps (e.g., Execute Shell) to install dependencies and run tests:

```
npm install
npm test
```

4. Writing Jenkinsfile to Automate CI Processes:

- **Create a `Jenkinsfile`** in the root of the Node.js project repository to define the CI stages:

```
pipeline {
  agent any

  stages {
    stage('Install Dependencies') {
      steps {
        sh 'npm install'
      }
    }
    stage('Run Tests') {
      steps {
        sh 'npm test'
      }
    }
  }
}
```

- This pipeline will install project dependencies and run unit tests using Jest or Mocha.

5. Running the Pipeline:

- Commit the `Jenkinsfile` to your Git repository.
- Create a new pipeline in Jenkins, linking it to the repository.
- Jenkins will automatically detect the `Jenkinsfile` and run the defined stages.

Hands-On Implementation: CI/CD Pipeline for a Node.js Application with Jenkins

Step 1: Install Jenkins and Node.js on Local Machine

- Follow the instructions provided above to install Jenkins and Node.js on your local machine or server.

Step 2: Create a Simple Node.js Application

- Create a basic Node.js project using `npm init` and install Express.js for server-side functionality.

```
npm init -y
npm install express

Create a simple app.js file with the following code:

const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello World');
});

app.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

Step 3: Writing Unit Tests with Jest and Mocha

- **Install Jest** for unit testing:

```
npm install --save-dev jest
```

Add a test script to `package.json`:

```
npm install --save-dev mocha chai
```

Alternatively, **Install Mocha** and Chai for flexible testing:

```
npm install --save-dev mocha chai
```

Write unit tests to verify the functionality of your Express.js application.

- **Example test file for Jest** (`app.test.js`):

```
const request = require('supertest');
const app = require('./app');

describe('GET /', () => {
  it('should return Hello World', async () => {
    const res = await request(app).get('/');
    expect(res.text).toBe('Hello World');
    expect(res.statusCode).toBe(200);
  });
});
```

Example test file for Mocha (`test/app.test.js`):

```
const request = require('supertest');
const chai = require('chai');
const expect = chai.expect;
const app = require('../app');

describe('GET /', () => {
  it('should return Hello World', async () => {
    const res = await request(app).get('/');
    expect(res.text).to.equal('Hello World');
```

```
    expect(res.statusCode).to.equal(200);
  });
});
```

Step 4: Configure Jenkins to Run the CI Pipeline

- Commit your project (including the `Jenkinsfile`) to GitHub.
- In Jenkins, create a new pipeline and link it to your GitHub repository.

Step 5: Running the Pipeline on Jenkins

- Jenkins will detect the `Jenkinsfile` and automatically run the CI stages for every commit.

Step 6: Review Test Reports in Jenkins

- Once the pipeline finishes, review the test reports and logs to ensure the application is functioning correctly.

Step 7: Deploying the Application

- You can add further stages in the `Jenkinsfile` to deploy the Node.js application to a server or platform of choice after successful testing.

Step 8: Moving to Cloud (Optional)

- Set up Jenkins on a cloud instance (AWS EC2) and configure the same pipeline for scalability and production deployment.

Best Practices in Continuous Integration (CI)

Continuous Integration (CI) is a crucial practice in modern software development, enabling teams to detect and address issues early in the development process. By following best practices, organizations can maximize the efficiency and effectiveness of their CI pipelines. Below are some of the best practices to consider:

1. Commit Code Frequently

- **Rationale**: Frequent code commits ensure that changes are integrated regularly, reducing the complexity of merging large changes and minimizing the risk of integration conflicts.
- **Benefits**:
    - **Smaller, Manageable Changes**: Regular commits lead to smaller code changes, making it easier to identify the source of issues.
    - **Continuous Feedback**: Developers receive feedback sooner, allowing them to address problems quickly.

2. Maintain a Fast Build Process

- **Rationale**: A fast build process is essential for maintaining the flow of development. If builds take too long, they can slow down the entire development process and discourage frequent commits.
- **Strategies**:
  - **Optimize Build Steps**: Identify and eliminate bottlenecks in the build process.
  - **Parallelize Builds**: Where possible, parallelize build steps to reduce overall build time.
  - **Incremental Builds**: Implement incremental builds that only rebuild the parts of the project that have changed.

3. Keep the Build Environment Consistent

- **Rationale**: Consistency across development, testing, and production environments helps avoid “it works on my machine” issues. Variations in environment configurations can lead to unexpected behavior when deploying applications.
- **Best Practices**:
  - **Environment as Code**: Use tools like Docker or Vagrant to define and manage environments as code.
  - **Automated Configuration Management**: Tools like Ansible, Chef, or Puppet can help ensure that all environments are configured consistently.

4. Automate Testing and Integration as Much as Possible

- **Rationale**: Automation is at the heart of CI. Automating testing and integration tasks reduces the manual effort required, improves accuracy, and allows the CI pipeline to run frequently and reliably.
- **Key Areas to Automate**:
  - **Unit Testing**: Automate the execution of unit tests to verify that individual components of the application function as expected.
  - **Integration Testing**: Automate integration tests to ensure that different parts of the application work together correctly.
  - **Code Quality Checks**: Automate code quality checks using tools like SonarQube to maintain code standards.
  - **Deployment**: Automate the deployment process to ensure that the application is consistently and reliably deployed to the target environment.

5. Monitor and Maintain the CI Pipeline

- **Rationale**: Continuous monitoring and maintenance of the CI pipeline ensure its reliability and effectiveness. A well-maintained pipeline can adapt to changes in the project and environment.
- **Best Practices**:
  - **Pipeline Health Checks**: Regularly monitor the health of the pipeline to detect and resolve issues promptly.
  - **Update Tools and Dependencies**: Keep the CI tools and dependencies up to date to benefit from new features and security patches.
  - **Feedback Loops**: Implement feedback loops that notify developers of any issues in the CI process, enabling them to take corrective action quickly.

6. Secure the CI Pipeline

- **Rationale**: Security is an often-overlooked aspect of CI. A secure CI pipeline helps protect the codebase from unauthorized access and ensures the integrity of the build process.
- **Security Best Practices**:
    - **Access Control**: Restrict access to the CI system and repositories to authorized personnel only.
    - **Secure Dependencies**: Use tools to scan dependencies for known vulnerabilities and keep them up to date.
    - **Audit Logs**: Implement audit logging to track changes and actions within the CI pipeline for accountability and traceability.
    - **Credential storage:** Store creds in Secrets in kubernetes or AWS KMS. Avoid hardcoding of creds in application code.